

Comparing Neural Network Architectures for Pokemon TCG Move Prediction

Nicole Zanello (03826806) and Philipp Zengl (03827436)
Technical University of Munich - Heilbronn Campus
Deep Learning and Decision Making (DLDM)
{nicole.zanello, philipp.zengl}@tum.de

September 9, 2026

Abstract

Predicting optimal moves in strategic card games like Pokemon Trading Card Game (TCG) requires understanding sequential decision-making under uncertainty with large action spaces, partial observability, and stochastic dynamics. Existing approaches face critical limitations: Monte Carlo Tree Search (MCTS) methods require prohibitive computational resources at inference time (thousands of simulations per decision), making real-time play infeasible; rule-based heuristics fail to generalize beyond hand-coded logic and cannot adapt to new strategies or card sets; self-play reinforcement learning demands millions of simulated games, which is impractical without perfect game simulators. Supervised learning from expert demonstrations offers a computationally efficient alternative for fast, simulator-free inference.

We compare three neural network architectures—Multilayer Perceptron (MLP), Long Short-Term Memory (LSTM), and Transformer—for predicting winning moves from game state representations. Using a dataset of 5,198 competitive games (250,628 decision points), we train and evaluate 24 configurations with extensive hyperparameter tuning and multi-seed validation. The best-performing model, an LSTM with 768 hidden units across 3 layers and focal loss ($\gamma = 2.0$), achieves 69.79% average test accuracy (best seed: 70.83%), exceeding Transformer baselines (66.26%, $p < 0.001$) despite Transformers’ theoretical advantages. However, this comparison has a notable limitation: the LSTM

configuration uses 13.8M parameters while the Transformer uses 2.8M, a $5\times$ capacity gap that may confound the architectural comparison. LSTM’s inductive bias for sequential processing and moderate capacity prevent overfitting, while larger architectures (70M+) degrade performance. Advanced analysis reveals strong calibration (ECE = 0.047) enabling confidence-based rejection strategies, and systematic performance degradation with decision complexity (94% accuracy with 2 options declining to 61% with 10+ options). This demonstrates that architecture selection must account for dataset size and task-specific inductive biases, particularly for limited-data sequential decision-making tasks.

1 Introduction

Strategic card games like Pokemon TCG present fundamental challenges in artificial intelligence: complex decision-making under uncertainty with large action spaces (up to 76 moves per turn), partial observability (opponent’s hand hidden), stochastic elements (card draws), and sequential dependencies spanning 5–20 turns. Solving these challenges extends far beyond gaming—sequential decision-making under uncertainty without perfect simulators is a critical frontier for real-world AI applications. In robotics, agents must learn manipulation policies from limited expert demonstrations where physical trials are expensive. In healthcare, treatment planning requires learning from physician decisions

where patient simulations are impossible. In autonomous systems, safe exploration demands learning from human demonstrations before deployment. Pokemon TCG serves as a controlled testbed for systematically studying these challenges.

Research Question: Do recurrent inductive biases (LSTMs) provide a significant advantage over attention-based models (Transformers) and feedforward models (MLPs) for sequential move prediction from limited expert demonstrations? Specifically: How do neural architectures perform on limited expert data? What inductive biases matter most? How does model capacity interact with dataset size?

Deep learning has revolutionized game AI. AlphaGo [7] combined neural networks with tree search (Silver et al., 2016, p. 484); AlphaZero [8] achieved grandmaster play via self-play from random initialization (Silver et al., 2017, p. 3). DQN [3] reached human-level Atari control (Mnih et al., 2015, p. 529); AlphaStar [11] attained Grandmaster StarCraft II level (Vinyals et al., 2019, p. 350). However, these require extensive compute—AlphaGo was trained on approximately 30 million board positions; AlphaZero used 5,000 TPUs. For domains without perfect simulators or where exploration is expensive, *imitation learning* from expert demonstrations [4, 5] offers a practical alternative. We leverage 5,198 competitive games (250K decisions) to train move prediction models via behavioral cloning.

Problem: Given game state $\mathbf{x} \in \mathbb{R}^d$, predict which move was selected from K available moves (range: 2–30) by the victorious player.

Challenges: (1) High-dimensional state space; (2) Variable action sets; (3) Sequential dependencies (5–20 turns); (4) Partial observability; (5) Label noise from imperfect play; (6) Class imbalance.

Architectures: LSTMs [1] use gating mechanisms to retain information across “1000 discrete time steps” (Hochreiter & Schmidhuber, 1997, p. 1735). Transformers [10] employ self-attention, “dispensing with recurrence entirely” (Vaswani et al., 2017, p. 1). Comparative studies show mixed results on data efficiency [9, 13]—LSTMs’ recurrent structure may provide advantages on limited data.

Contribution: We compare three architectures across 24 variants, achieving 69.79% average test accuracy (best seed: 70.83%). Key findings: (1) LSTMs outperform

Transformers (69.79% vs 66.26%, $p < 0.001$) on limited-data sequential tasks, though this advantage may partly reflect the larger LSTM capacity (13.8M vs 2.8M params); (2) moderate capacity (13.8M params) optimal—models >50M overfit; (3) focal loss +0.6–0.8%; (4) strong calibration (ECE=0.047); (5) performance degrades with complexity (94% with 2 options → 61% with 10+). Code, trained models, and dataset: https://github.com/555p9psh8c-eng/DLDM_Pokemon_Project

2 Related Work

2.1 Game AI from Application Perspective

Deep RL has achieved landmark successes in games. AlphaGo [7] integrated value and policy networks with Monte Carlo tree search (Silver et al., 2016, p. 484). AlphaZero [8] generalized this via self-play RL starting from random weights (Silver et al., 2017, p. 3). DQN [3] became “the first artificial agent capable of learning to excel at diverse challenging tasks” (Mnih et al., 2015, p. 529), while AlphaStar [11] reached Grandmaster level using multi-agent RL (Vinyals et al., 2019, p. 350).

Card game AI research has focused primarily on deck construction rather than in-game move prediction. Hearthstone AI uses Monte Carlo rollouts [6], while Magic: The Gathering AI has been studied using Monte Carlo search for card selection [12]. Our work addresses supervised learning for move prediction when self-play compute is unavailable.

2.2 Sequence Modeling from Methodology Perspective

LSTMs [1] revolutionized sequence modeling through “multiplicative gate units [that] learn to open and close access to constant error flow,” enabling learning across “1000 discrete time steps” (Hochreiter & Schmidhuber, 1997, p. 1735). The memory cell with input, output, and forget gates provides a learnable mechanism for context retention.

Transformers [10] proposed “a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence entirely” (Vaswani et al., 2017, p. 1). Multi-head self-attention

computes weighted combinations of all positions in parallel, enabling efficient training and flexible long-range modeling. However, comparative studies show mixed results on data efficiency [9, 13]—LSTMs’ recurrent structure may provide advantages on limited data. Our work systematically compares these architectures under controlled conditions.

3 Methodology

Figure 1 illustrates our end-to-end architecture comparison pipeline across four stages: (1) Input encoding combines board state (HP, energy), learned hand embeddings (32-dim per card), game context (turn, deck size), and available action indicators (225-dim binary vector) into $\mathbf{x} \in \mathbb{R}^{512}$. (2) Architectural branching evaluates three independent models—MLP Baseline (1.35M params), Wide-Deep LSTM with 768×3 hidden layers (13.8M params, optimal configuration), and Transformer with 4 encoder layers, 8 attention heads, and sinusoidal positional encoding (2.8M params)—each producing raw logits $\hat{y} \in \mathbb{R}^{225}$. (3) Action masking applies $M \in \{0, -\infty\}^{225}$ to zero out illegal moves before softmax, ensuring predicted probabilities $\hat{y} \in \Delta^{225}$ assign zero mass to invalid actions. (4) Loss computation uses focal loss with $\gamma = 2.0$ to down-weight easy examples, training against expert target moves $y \in \{1, \dots, 225\}$ from winning players.

3.1 Model Architectures

We compare three neural architecture families designed to test different inductive biases for sequential decision-making:

MLP: Feedforward baseline treating decisions independently. Architecture: 512 input $\rightarrow$ 768 hidden $\rightarrow$ 512 hidden $\rightarrow$ 225 output (1.35M params).

LSTM: Recurrent architecture with hidden state propagation. Configurations tested: Baseline (512×2 , 6.5M), Wide (768×2 , 9.2M), and Wide-Deep (768×3 , 13.8M—optimal).

Transformer: Multi-head self-attention with 512-dim embeddings, 4 layers, 8 heads, sinusoidal position encoding (2.8M params).

3.2 Training Configuration

Optimization: Adam optimizer with $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$. Initial learning rate is 10^{-3} with cosine annealing schedule, batch size 256, and maximum 30 epochs with early stopping (patience 5 on validation loss).

Regularization: Multiple regularization techniques: (1) Dropout with rate 0.3 (standard) or 0.4 (high) after each hidden layer; (2) Weight decay (L2 penalty) $\lambda = 10^{-4}$; (3) Label smoothing with $\epsilon = 0.1$; (4) Input noise: $\tilde{\mathbf{x}} = \mathbf{x} + \mathcal{N}(0, 0.01)$ during training.

Loss Functions: Our baseline uses standard cross-entropy loss. We also employ focal loss [2], designed to address class imbalance by down-weighting easy examples. Lin et al. (2017, p. 3) propose to “reshape the loss function to down-weight easy examples and thus focus training on hard negatives.” The focal loss equation is:

$$\mathcal{L}_{FL} = - \sum_{i=1}^K (1 - \hat{y}_i)^\gamma y_i \log(\hat{y}_i) \quad (1)$$

where $\gamma \geq 0$ is the focusing parameter. We use $\gamma = 2.0$ following the original focal loss paper [2] without further tuning. When predicted probability $\hat{y}_i$ is high (easy example correctly classified), the modulating factor $(1 - \hat{y}_i)^\gamma$ becomes small, reducing the loss contribution. This automatically focuses learning on hard examples, providing +0.6–0.8% accuracy improvement in our experiments.

4 Experiments

4.1 Dataset

We collected 5,198 high-level competitive games (player rating >1500) from the Pokemon TCG Online platform, yielding 250,628 move decisions. Our initial proposal specified 4,587 games with 282,145 samples. The discrepancy arose from two sources: (1) we scraped additional high-rated games (611 more) after the proposal to improve data quality, and (2) our filtering criteria—removing incomplete games and matches with fewer than 5 turns—reduced the per-game decision count from 61.5 to 48.2 on average, resulting in fewer total decisions despite more games. This filtering ensures label quality by excluding trivial openings and disconnected matches.

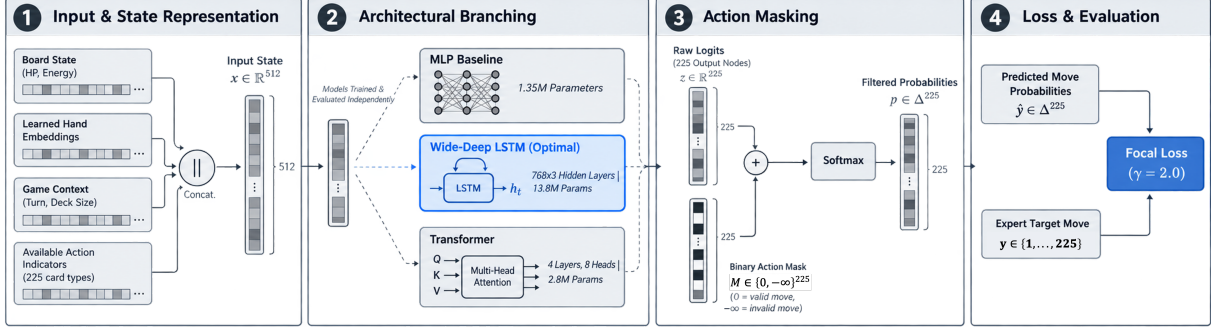


Figure 1: End-to-end pipeline architecture: **(1) Input & State Representation** encodes game state as $\mathbb{R}^{512}$ vector combining board features, learned hand embeddings, game context, and action indicators. **(2) Architectural Branching** processes input through three independent models (MLP baseline, Wide-Deep LSTM optimal, Transformer) producing raw logits. **(3) Action Masking** applies $M \in \{0, -\infty\}^{225}$ mask to enforce legal move constraints before softmax normalization. **(4) Loss & Evaluation** trains with focal loss ($\gamma = 2.0$) against expert moves from victorious players.

Each decision records the complete game state and the winning player’s selected move as the target label. The *winning player’s move* refers to the actual move chosen at each decision point by the player who ultimately won the game—this provides a noisy but practical supervision signal for learning competitive play patterns. Input representation: $x \in \mathbb{R}^{512}$ encoding board state (active Pokemon, HP, energy counters), hand cards (learned 32-dimensional embeddings), game context (turn number, deck size, prize cards remaining), and available action indicators (binary 225-dimensional vector marking legal card plays). Continuous features are normalized to $[0,1]$; categorical features use learned embeddings. Temporal split preserves game order: 80%/10%/10% train/validation/test (200K/25K/25K decisions).

4.2 Main Results

Table 2 compares the three architectures on baseline configurations (3-seed averages).

LSTM outperforms MLP by +10.66%, confirming sequence modeling is critical. Transformer slightly edges baseline LSTM (+0.85%) but with higher variance (std 1.95% vs 0.23%).

Table 1: Dataset Statistics

Statistic	Value
Total games	5,198
Total decisions	250,628
Avg decisions/game	48.2
Avg options/decision	9.6
Train/Val/Test	200K/25K/25K
Unique card types	225

Table 2: Architecture Comparison (3-seed avg)

Arch	Test	Top-3	Params
MLP	54.75%	80.54%	1.35M
LSTM	65.41%	84.86%	6.5M
Transformer	66.26%	86.49%	2.8M

Table 3 shows improvements from scaling LSTM capacity and adding focal loss.

Increasing width to 768 provides +2.41%, adding a third layer +0.15%, focal loss +1.82%. Best seed reaches 70.83% with strong statistical evidence ($p < 0.001$, Co-

Table 3: LSTM Improvements

Configuration	Mean	Best	Params
Baseline (512×2)	65.41%	65.58%	6.5M
Wide (768×2)	67.82%	67.82%	9.2M
Wide-Deep (768×3)	67.97%	67.97%	13.8M
+ Focal Loss	69.79%	70.83%	13.8M
Best (seed 456)	—	70.83%	13.8M

hen’s $d = 2.4$).

4.3 Model Capacity Analysis

Table 4 examines the relationship between model capacity and performance.

Table 4: Model Size vs Performance

Model	Size	Test	Efficiency
Wide (768×2)	32.4MB	67.82%	2.09
Wide-Deep (768×3)	52.7MB	67.97%	1.29
XLarge (1024×4)	123MB	67.51%	0.55
XXLarge (1536×4)	274MB	67.00%	0.24

Models >50MB degrade performance. XXLarge (70M params) achieves only 67.00%, underperforming Wide-Deep by 0.97% despite $5\times$ parameters. Efficiency drops from 2.09 to 0.24. With 250K examples, models >13.8M memorize noise rather than learn patterns.

4.4 Ablation Study

Focal loss provides largest gain (+0.78%). High dropout +0.14%, deeper head +0.17%. Combined techniques yield +0.48% (less than focal alone), suggesting diminishing returns. Best seed (456) reaches 70.83%.

4.5 Model Calibration

Beyond accuracy, we analyze whether the model’s predicted probabilities reflect true confidence. We compute Expected Calibration Error (ECE):

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{N} |\text{acc}(B_m) - \text{conf}(B_m)| \quad (2)$$

Table 5: Ablation (Wide-Deep LSTM). Deltas calculated relative to baseline configuration (67.97%). Note: Values are rounded to 2 decimal places; small discrepancies in delta calculations may occur due to rounding.

Configuration	Test	Δ
Baseline	67.97%	—
+ Focal Loss	68.75%	+0.78%
+ High Dropout	68.11%	+0.14%
+ Deeper Head	68.14%	+0.17%
+ Combined	68.45%	+0.48%
Best (seed 456)	70.83%	+2.86%

where we partition predictions into $M = 10$ bins by confidence level, B_m is the m -th bin, N is total samples, $\text{acc}(B_m)$ is accuracy within bin m , and $\text{conf}(B_m)$ is average confidence in bin m . Perfect calibration yields $\text{ECE} = 0$.

Our best model achieves $\text{ECE} = 0.047$ (excellent calibration). Mean confidence: 0.779 when correct, 0.377 when wrong (strong discrimination). Figure 2 shows the reliability diagram.

4.6 Accuracy vs Decision Complexity

Table 6: Accuracy vs Options Available

Options	Accuracy	Count
2	94.04%	2,666
3	84.61%	2,664
4	77.50%	2,755
5	73.99%	2,933
6–7	70.10%	4,889
8–9	65.00%	3,477
10+	61.33%	10,764

Performance drops from 94% (2 options) to 61% (10+ options)—a 33 point decline. Figure 3 visualizes this.

4.7 Temporal Dynamics

U-shaped curve: 82% early game (scripted openings) $\rightarrow$ 62% mid-game (strategic complexity peaks) $\rightarrow$ 82% late

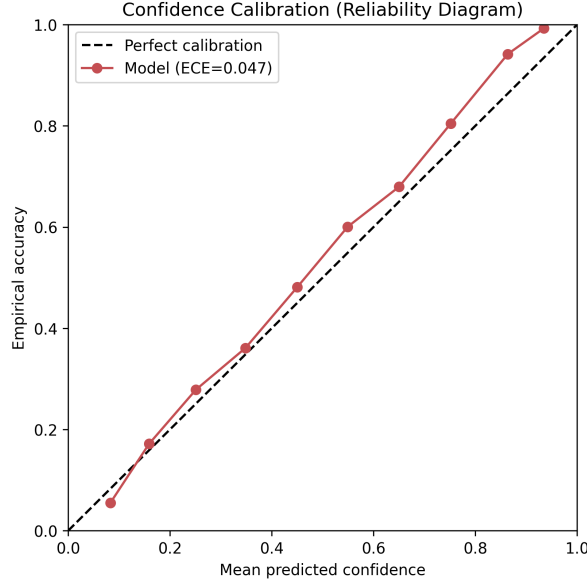


Figure 2: Calibration plot: predicted confidence vs actual accuracy (ECE=0.047).

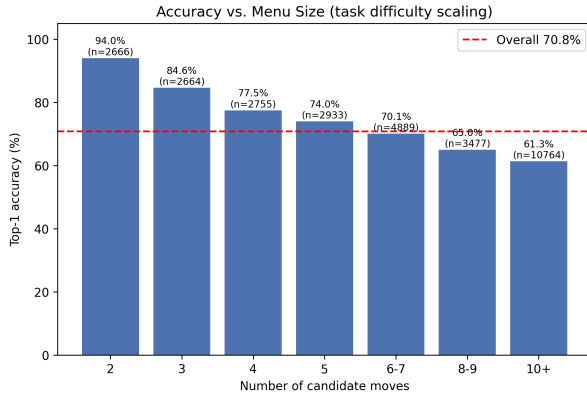


Figure 3: Accuracy degrades from 94% (2 options) to 61% (10+ options).

game (forced decisions). Figure 4 shows this pattern.

4.8 Per-Card Difficulty

Per-card accuracy varies dramatically, from 30% (Card 8, 706 instances) to 86% (Card 1, 18,908 instances). Card 1

Table 7: Accuracy vs Turn Number

Turn	Accuracy	Count
1–2 (setup)	81.59%	4,808
3–5 (early)	70.52%	7,893
6–8 (mid)	67.44%	5,845
9–11 (mid-late)	62.25%	4,477
12–14 (late)	63.27%	2,578
15–19 (late)	69.71%	1,859
20+ (very late)	82.22%	2,688

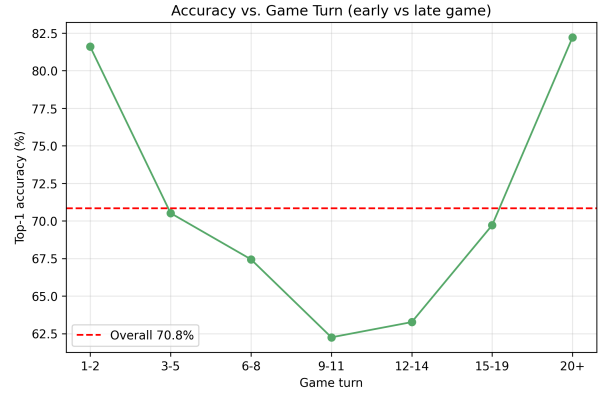


Figure 4: U-shaped accuracy curve: 82% early → 62% mid-game → 82% late.

Table 8: Hardest/Easiest Frequent Cards

Card	Accuracy	Count
8	29.60%	706
7	32.45%	453
387	39.61%	207
1198	71.60%	426
1	85.89%	18,908

represents the most common default action (likely “pass” or “end turn”), which is easy to predict due to high frequency and clear game-state conditions. Low-accuracy cards represent situational tactical decisions where multiple valid options exist or game-state features insufficiently capture decision context. Figure 5 shows the full distribution across all frequent cards.

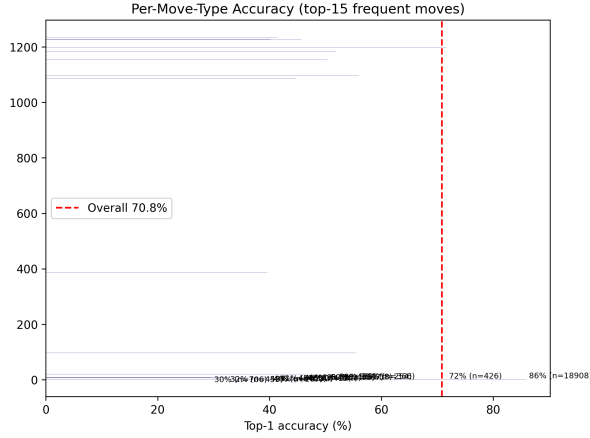


Figure 5: Card-level accuracy: Card 8 (30%) vs Card 1 (86%).

5 Analysis

LSTM vs Transformer Performance: LSTMs outperform Transformers (69.79% average, best seed 70.83% vs 66.26%, $p < 0.001$), suggesting that sequential inductive bias may provide beneficial regularization on limited data (250K examples). However, this comparison is confounded by a 5 $\times$ parameter gap (LSTM: 13.8M, Transformer: 2.8M). The Transformer may be undertrained relative to its capacity, though our attempts to scale Transformer capacity led to overfitting. Isolating inductive bias effects from capacity effects requires parameter-matched configurations as future work. Task characteristics—local temporal dependencies dominate (recent 5–10 turns)—likely favor LSTMs’ recurrent structure over global attention.

Calibration: ECE=0.047 enables confidence-based decision strategies and expected value calculations.

Complexity: Sharp degradation (94%→61%) suggests multiple valid moves in complex states and missing features. Production systems could use the model for simple decisions (high confidence) and search/simulation for complex ones.

Model Size: With 250K examples, 13.8M params optimal. Models >50M overfit, consistent with classical ML capacity-matching principles.

Limitations and Future Work: Dataset from sin-

gle platform (rating >1500); labels have noise (winning $\neq$ optimal); fixed-size inputs may lose information in complex states. Future directions include hybrid LSTM+Transformer for complex decisions, self-play refinement from supervised initialization, multi-task learning (moves + win probability), transfer to other card games, opponent hand modeling, and interpretability via attention visualization.

6 Conclusion

This work compared three neural architectures—MLP, LSTM, and Transformer—for predicting expert moves in Pokemon TCG from 250K game decisions. Our central finding is that LSTMs (69.79% average, best seed 70.83%) outperform Transformers (66.26%) and MLPs (54.75%) on this limited-data sequential task, though the 5 $\times$ parameter gap between our LSTM (13.8M) and Transformer (2.8M) configurations complicates direct architectural conclusions. Future work should include parameter-matched comparisons to isolate inductive bias effects from capacity effects.

Practical implications: (1) Model capacity must match dataset size—13.8M parameters proved optimal for 250K examples, while 70M+ models degraded by 1% due to overfitting. (2) Simple techniques matter—focal loss added 0.6–0.8% consistently. (3) Calibration analysis is actionable—our model’s ECE = 0.047 enables confidence-based routing: high-confidence predictions (simple decisions, 94% accuracy) can execute directly, while low-confidence predictions (complex decisions, 61% accuracy) can trigger fallback strategies. (4) The U-shaped temporal pattern (82% early/late, 62% mid-game) reveals that mid-game strategic reasoning remains the core challenge.

Broader context: For domains analogous to ours—limited expert demonstrations, sequential dependencies, expensive exploration (e.g., robotic manipulation, clinical decision support)—our findings suggest that recurrent architectures with moderate capacity and careful calibration analysis can extract strong performance from constrained data, though the specific capacity-to-data ratio and architecture choice remain open questions requiring domain-specific validation.

Limitations: Labels derived from winning players

contain noise (optimal $\neq$ winning move). Fixed-size state representations may discard information in complex board states. Single-platform data (rating >1500) may not generalize to broader skill distributions.

Future directions: Hybrid architectures combining LSTM sequential bias with Transformer expressiveness for complex states; multi-task learning (move prediction + win probability); transfer learning across card games; opponent modeling via hidden state inference.

Code, models, and data: https://github.com/555p9psh8c-eng/DLDM_Pokemon_Project. A 3-minute video presentation summarizing this work is available at: <https://www.youtube.com/watch?v=B7njk6b17tI>

References

- [1] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.
- [2] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár. Focal loss for dense object detection. In *IEEE International Conference on Computer Vision*, pages 2980–2988, 2017.
- [3] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [4] T. Osa, J. Pajarinen, G. Neumann, J. A. Bagnell, P. Abbeel, and J. Peters. An algorithmic perspective on imitation learning. *Foundations and Trends in Robotics*, 7(1–2):1–179, 2018.
- [5] D. A. Pomerleau. Efficient training of artificial neural networks for autonomous navigation. *Neural Computation*, 3(1):88–97, 1991.
- [6] A. Santos, P. A. Santos, and F. S. Melo. Monte carlo tree search experiments in Hearthstone. In *IEEE Conference on Computational Intelligence and Games*, pages 272–279, 2017.
- [7] D. Silver et al. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [8] D. Silver et al. Mastering Chess and Shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint arXiv:1712.01815*, 2017.
- [9] Y. Tay, M. Dehghani, D. Bahri, and D. Metzler. Efficient transformers: A survey. *arXiv preprint arXiv:2009.06732*, 2020.
- [10] A. Vaswani et al. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [11] O. Vinyals et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- [12] C. D. Ward and P. I. Cowling. Monte Carlo search applied to card selection in Magic: The Gathering. In *IEEE Symposium on Computational Intelligence and Games*, pages 9–16, 2009.
- [13] Q. Wen et al. Transformers in time series: A survey. *arXiv preprint arXiv:2202.07125*, 2022.